

Tổng Hợp Kiến Thức

I. Tại Sao & Khi Nào Cần Tối Ưu?

- **Mục đích:** Cải thiện Trải nghiệm Người dùng (UX), Tốc độ tải trang, Tỷ lệ chuyển đổi, SEO.
- **Vấn đề phổ biến:**
 - **Large Initial Bundle:** File JS lớn tải ban đầu → Chậm Time-to-Interactive (TTI).
 - **Unnecessary Re-renders:** Component re-render lại dư thừa dù không có gì thay đổi → Lãng phí CPU, có thể gây giật lag.
- **Nguyên tắc:**
 - **ĐO LƯỜNG TRƯỚC KHI TỐI ƯU! (Measure First!)**
 - **Tránh Tối Ưu Quá Sớm (Avoid Premature Optimization).**
 - Tối ưu đi kèm chi phí (độ phức tạp code).

II. Công Cụ Đo Lường

1. React DevTools - Profiler Tab:

- **Mục đích:** Ghi lại và phân tích các lần render.
- **Tìm:** Component nào render lại thường xuyên? Component nào render tốn thời gian? Tại sao component re-render?
- **Cách dùng:** Record → Tương tác với UI → Stop → Phân tích (Flamegraph, Ranked chart).

2. Browser DevTools - Network Tab:

- **Mục đích:** Kiểm tra kích thước file JS, xem xét hiệu quả Code Splitting.
- **Tìm:** Kích thước bundle ban đầu? Các chunk code có được tải động khi điều hướng không?

III. Kỹ Thuật Tối Ưu Chính

1. Code Splitting (Giảm Bundle Ban Đầu)

- **Kỹ thuật:** Chia bundle lớn thành các chunk nhỏ, tải theo yêu cầu.
- **Công cụ:**
 - `React.lazy(() => import('./MyComponent'))` :
 - Dùng để import động component (chỉ hoạt động với `default export`).
 - Trả về component có thể render như bình thường.
 - `<Suspense fallback={<LoadingIndicator />}>` :
 - Bọc component được lazy load.
 - Hiển thị `fallback` UI trong khi component đang tải.
- **Áp dụng:** Thường dùng cho Route-based code splitting (tải code của trang chỉ khi truy cập).

```
// App.jsx
import React, { Suspense, lazy } from 'react';

const HomePage = lazy(() => import('./screens/Home'));
const LoginPage = lazy(() => import('./screens/Login'));

function App() {
  return (
    <Suspense fallback={<div>Loading...</div>}>
      <Routes>
        <Route path="/" element={<HomePage />} />
        <Route path="/login" element={<LoginPage />} />
      </Routes>
    </Suspense>
  );
}
```

```
    </Routes>
  </Suspense>
);
}
```

2. Ngăn Re-render Không Cần Thiết (**React.memo**)

- **Kỹ thuật:** Ghi nhớ (memoize) kết quả render của functional component.
- **Công cụ:** `React.memo(MyFunctionalComponent)` (Higher-Order Component)
 - Component chỉ re-render nếu props của nó **thay đổi** (so sánh nông - shallow comparison).
- **Shallow Comparison:**
 - Primitives (string, number, boolean): So sánh giá trị.
 - Objects, Arrays, Functions: So sánh **tham chiếu (reference)**. `{} !== {}` , `[] !== []` , `() ⇒ {} !== () ⇒ {}` .
- **Áp dụng:** Bọc các component:
 - Thường xuyên re-render không cần thiết (do cha re-render).
 - Có quá trình render tốn kém.
 - Nhận props ít thay đổi.

```
// TodoItem.jsx
import React from 'react';

const TodoItem = React.memo(function TodoItem({ todo /*, other props */ }) {
  console.log('Rendering TodoItem:', todo.id);
  // ... component logic
});

export default TodoItem;
```

- **Vấn đề:** `React.memo` không hiệu quả nếu props là functions/objects/arrays được tạo mới mỗi lần render của cha
→ Cần `useCallback` & `useMemo` .

3. Ổn Định Tham Chiếu (**useCallback** & **useMemo**)

- **Mục đích:** Giữ nguyên tham chiếu của functions và objects/arrays giữa các lần render (nếu dependencies không đổi), giúp `React.memo` hoạt động hiệu quả.
- **`useCallback(fn, [deps])` :**
 - Memoize **hàm callback (fn)**.
 - Trả về cùng một tham chiếu hàm nếu `deps` không đổi.
 - **Dùng cho:** Hàm truyền xuống component con đã bọc `React.memo` .

```
// ParentComponent.jsx
import React, { useState, useCallback } from 'react';
import MemoizedChild from './MemoizedChild';

function ParentComponent() {
  const [count, setCount] = useState(0);
  // Hàm này sẽ giữ nguyên tham chiếu trừ khi count thay đổi (ví dụ không thực tế)
  // Thường dùng cho các hàm không phụ thuộc state thay đổi thường xuyên, hoặc dùng functional update
  const handleClick = useCallback(() ⇒ {
    console.log('Button clicked!');
    // Cần đưa dependency nếu dùng giá trị ngoài (trừ setter state)
  }, []); // Dependency array rỗng nếu không dùng giá trị ngoài
```

```
return <MemoizedChild onClick={handleClick} />;
}
```

- `useMemo(() ⇒ computeValue(a, b), [a, b])` :
 - Memoize **kết quả của hàm** (`computeValue`).
 - Chỉ tính toán lại giá trị nếu `deps` thay đổi.
 - **Dùng cho:**
 - Tính toán tốn kém.
 - Tạo object/array phức tạp để truyền làm prop cho component con đã bọc `React.memo` .

```
// ParentComponent.jsx
import React, { useState, useMemo } from 'react';
import MemoizedChild from './MemoizedChild';

function ParentComponent() {
  const [filter, setFilter] = useState('');
  const [items, setItems] = useState([...]);

  const filteredItems = useMemo(() ⇒ {
    console.log('Filtering items...');
    return items.filter(item ⇒ item.name.includes(filter));
  }, [items, filter]); // Chỉ lọc lại khi items hoặc filter thay đổi

  const styleObject = useMemo(() ⇒ ({
    color: 'blue',
    fontWeight: 'bold'
  }), []); // Tạo object style ổn định

  return <MemoizedChild data={filteredItems} style={styleObject} />;
}
```

- **Lưu ý:**
 - **Dependencies (`deps`) là quan trọng!** Phải liệt kê *tất cả* các giá trị (state, props, ...) từ scope ngoài mà hàm/tính toán sử dụng. Bỏ sót → bug. Thêm thừa → mất tác dụng memoize.
 - `useCallback(fn, deps)` tương đương `useMemo(() ⇒ fn, deps)` .
 - Không lạm dụng, chỉ dùng khi có lợi ích rõ ràng (đo lường được).

IV. Kỹ Thuật Khác (Tham khảo)

- **Virtualization (Virtualized Lists):**
 - **Khi nào:** Danh sách rất dài (hàng trăm/ngày items).
 - **Cách hoạt động:** Chỉ render các item đang hiển thị trong viewport (+ đệm).
 - **Thư viện:** `react-window` , `react-virtualized` .